

实验二：基于 ViT 的 CIFAR10 图像分类

尧祥临 202518023406038 前沿交叉科学学院

1 任务介绍

本次实验的任务是基于 Vision Transformer (ViT) 模型完成 CIFAR10 图像分类。实验指导书要求从零开始完成数据读取、网络构建、模型训练和模型测试等过程，并在 CIFAR10 数据集的测试集上达到 80% 以上的分类准确率。

CIFAR10 数据集包含 60000 张 32×32 彩色图像，共 10 个类别，分别为 airplane、automobile、bird、cat、deer、dog、frog、horse、ship 和 truck。官方训练集包含 50000 张图像，测试集包含 10000 张图像。本实验将官方训练集划分为 45000 张训练样本和 5000 张验证样本，官方测试集保持为 10000 张，用于最终评估。

2 模型架构

本实验实现的是适配 CIFAR10 小尺寸图像的 ViT 模型。由于 CIFAR10 图像尺寸为 32×32 ，这里没有将图像放大到标准 ViT 常用的 224×224 ，而是直接在原始分辨率上进行 patch 划分。模型整体由 Patch Embedding、位置编码、Transformer Encoder 和分类头组成。

2.1 Patch Embedding

输入图像大小为 $3 \times 32 \times 32$ ，patch 大小设置为 4×4 ，因此每张图像会被划分为

$$N = \left(\frac{32}{4}\right)^2 = 64$$

个 patch。代码中使用一个卷积层完成 patch 划分和线性投影，其卷积核大小和步长都等于 patch 大小，因此卷积输出为 $256 \times 8 \times 8$ ，展平并转置后得到 64×256 的 token 序列。

```
1 class Embedding(nn.Module):
2     def __init__(self, image_size=32, patch_size=4,
3                   in_channels=3, embed_dim=256):
4         super().__init__()
5         self.num_patches = (image_size // patch_size) ** 2
6         self.embedding = nn.Sequential(
7             nn.Conv2d(in_channels, embed_dim,
8                       kernel_size=patch_size,
9                       stride=patch_size,
10                      bias=False),
11             nn.Flatten(2)
12         )
13
```

```

14 def forward(self, x):
15     output = self.embedding(x)
16     output = output.transpose(1, 2)
17     return output

```

2.2 Transformer Encoder

每个 Transformer 层由多头自注意力模块、MLP 模块、LayerNorm、残差连接和 DropPath 组成。注意力模块使用 PyTorch 中的 `nn.MultiheadAttention` 实现，并设置 `batch_first=True`，使输入维度保持为 $B \times N \times D$ 。MLP 模块由两层全连接层、GELU 激活函数和 Dropout 组成，隐藏层维度为 1024。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

在本模型中，嵌入维度 $D = 256$ ，注意力头数为 8，因此每个 head 的维度为 32。Transformer 深度设置为 8，即堆叠 8 个编码器层。

```

1 class Transformer(nn.Module):
2     def __init__(self, embed_dim=256, num_heads=8,
3                 mlp_hidden_dim=1024, depth=8,
4                 dropout=0.1, drop_path_rate=0.1):
5         super().__init__()
6         drop_path_rates = torch.linspace(0, drop_path_rate, depth).tolist()
7         self.layers = nn.ModuleList([
8             nn.ModuleList([
9                 Attention(embed_dim, num_heads, dropout),
10                Mlp(embed_dim, mlp_hidden_dim, dropout),
11                DropPath(drop_path_rates[layer_idx]),
12                DropPath(drop_path_rates[layer_idx]),
13            ])
14            for layer_idx in range(depth)
15        ])
16
17    def forward(self, x):
18        for attn, mlp, attn_drop_path, mlp_drop_path in self.layers:
19            x = x + attn_drop_path(attn(x))
20            x = x + mlp_drop_path(mlp(x))
21        return x

```

2.3 整体结构

Patch token 前会拼接一个可学习的 `cls_token`，并加入可学习的位置编码。Transformer 输出后，代码没有直接使用 `cls_token` 做分类，而是对所有 patch token 取均值作为图像表示，再通过线性分

类头输出 10 类 logits。

表 1: ViT 模型主要结构

模块	输出形状	设置
Input	$3 \times 32 \times 32$	CIFAR10 彩色图像
Patch Embedding	64×256	patch size = 4, embed dim = 256
Token + Position	65×256	拼接 <code>cls_token</code> 并加入位置编码
Transformer Encoder	65×256	depth = 8, heads = 8
Mean Pooling	256	对 patch token 求均值
Classification Head	10	输出 10 类 logits

本模型总参数量为 6350346，其中 Transformer Encoder 约 6318080 个参数，是主要的参数来源。Patch Embedding 参数量为 12288，分类头参数量为 2570。

3 参数设置与训练方法

训练阶段采用 AdamW 优化器，并对不同参数组设置不同的权重衰减策略：对卷积和全连接权重使用 weight decay，对 bias、LayerNorm 参数、`cls_token` 和位置编码不使用 weight decay。这样做的原因是，归一化参数和位置相关参数本身并不直接承担普通线性权重的特征压缩作用，如果对他们施加同样的衰减，可能削弱位置编码和归一化层的表达能力。

相比卷积网络，ViT 缺少卷积结构中天然的局部平移归纳偏置，在 CIFAR10 这种小规模数据集上更容易出现过拟合。因此训练中加入了较强的数据增强和正则化策略：随机裁剪和随机水平翻转用于扩大输入扰动，Random Erasing 模拟局部遮挡，Mixup 和 label smoothing 降低模型对单个样本标签的过度自信，DropPath 约束深层 Transformer 的共适应。模型评估时使用 EMA 参数，可以降低单次梯度更新带来的参数震荡，使验证和测试指标更平稳。

学习率策略采用前 10 个 epoch 线性 warmup，随后进行余弦退火。warmup 阶段让 Transformer 在训练初期以较小步长进入稳定区域，避免注意力权重和 MLP 权重在随机初始化后被过大的学习率破坏；余弦退火则在中后期逐步减小学习率，使模型从快速拟合转向细粒度收敛。

表 2: 主要训练配置

项目	设置	项目	设置
Batch size	128	Optimizer	AdamW
初始学习率	4×10^{-4}	Weight decay	2×10^{-2}
Epochs	200	Early stopping	patience 40
Label smoothing	0.05	Mixup alpha	0.2
Drop path	0.1	EMA decay	0.9998
Gradient clip	1.0	Seed	114514

4 训练过程与实验结果

训练日志共记录 176 个 epoch。验证集准确率在第 136 个 epoch 达到最高值 85.68%，之后连续 40 个 epoch 没有进一步提升，因此训练过程触发 early stopping 并在第 176 个 epoch 停止。日志中测试集准确率最高为 85.79%，出现在第 131 个 epoch；验证集最优模型对应的测试集准确率为 85.61%；最后一轮测试集准确率为 85.57%。这些结果均明显高于实验指导书中 80% 的要求。

表 3: 关键训练节点

Epoch	Val Acc	Test Acc	说明
50	79.58%	80.59%	首次达到实验要求附近
100	84.58%	85.12%	模型进入稳定提升阶段
131	85.46%	85.79%	测试集准确率最高
136	85.68%	85.61%	验证集准确率最高
176	85.32%	85.57%	early stopping 停止训练

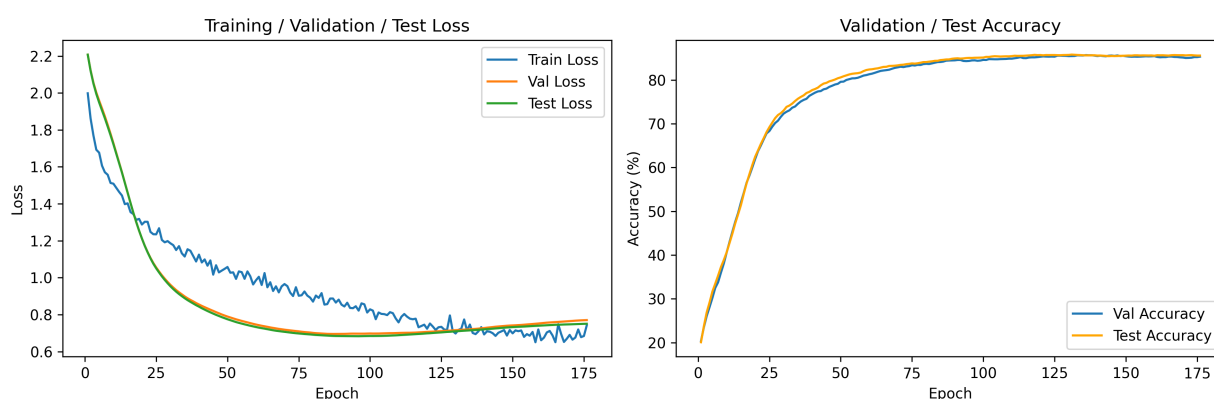


图 1: 训练损失与准确率变化曲线

从图 1 可以看到，训练前期 loss 下降较快，验证集和测试集准确率也快速提升。第 50 个 epoch 附近测试准确率已经达到 80.59%，超过实验要求；之后准确率继续缓慢提升，并在训练后期稳定在 85% 附近。这说明模型在前半段主要完成基础特征学习，后半段则更多是在正则化和学习率衰减的共同作用下进行小幅精修。

训练 loss 在后期仍有波动，但验证集和测试集曲线整体较平稳。这里的训练 loss 不能简单地与验证 loss 直接比较，因为训练阶段启用了 Mixup、Random Erasing 和随机裁剪，输入样本和标签都被扰动；验证和测试阶段则使用原始标签和确定性预处理。因此训练 loss 的波动更像是增强策略带来的优化噪声，而不是模型失效的信号。相反，验证集和测试集准确率在后期保持接近，说明模型没有出现明显的训练集记忆化问题。

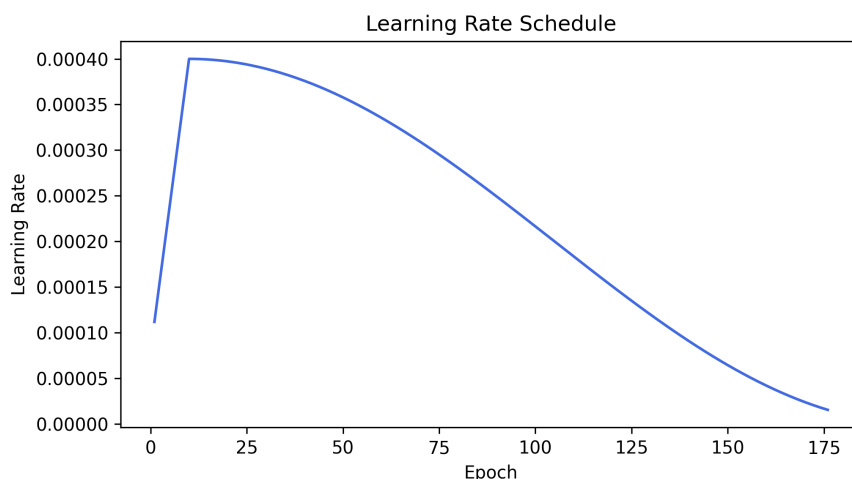


图 2: 学习率变化曲线

图 2 展示了学习率调度过程。前 10 个 epoch 为线性 warmup, 学习率从较小值逐步增加到 4×10^{-4} ; 随后进入余弦退火阶段, 学习率逐渐下降。该策略可以在训练早期避免模型参数剧烈震荡, 并在训练后期帮助模型更稳定地收敛。

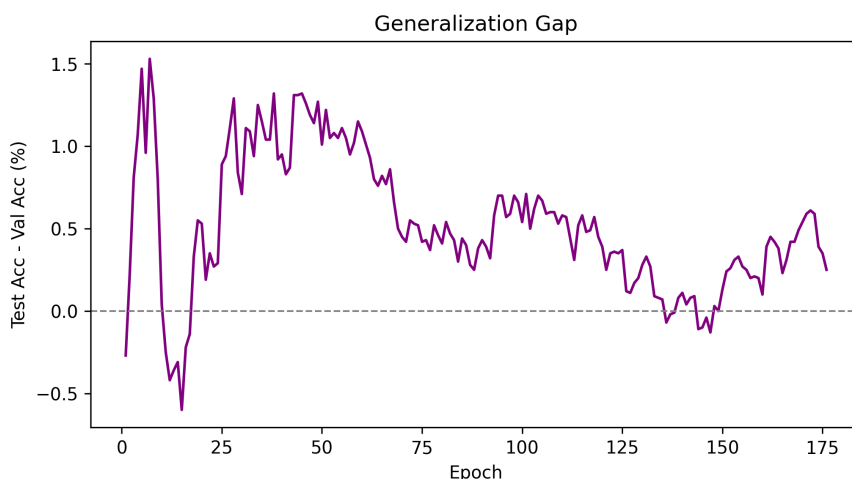


图 3: 测试集与验证集准确率差距

图 3 给出了测试集准确率与验证集准确率的差值。多数 epoch 中该差值都保持在较小范围内, 说明验证集能够较好反映测试集趋势, 没有出现验证集选择与测试集表现严重不一致的情况。第 131 个 epoch 测试集达到最高值, 而第 136 个 epoch 验证集达到最高值, 两者相差只有 5 个 epoch; 对应测试准确率分别为 85.79% 和 85.61%, 差距也很小。因此使用验证集最优模型作为最终提交模型是合理的。

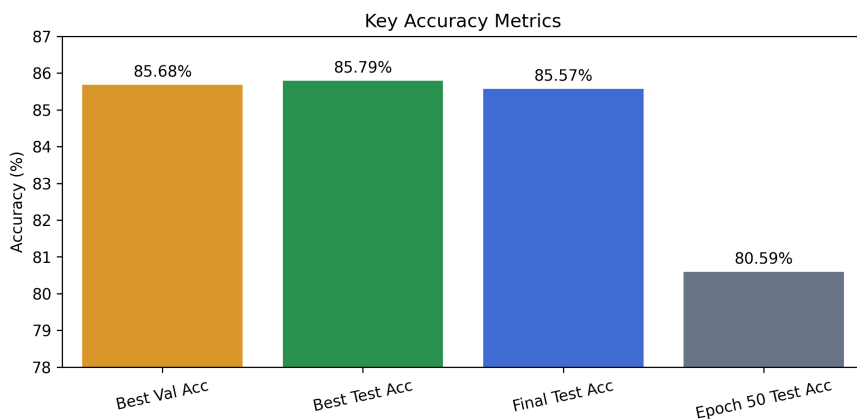


图 4: 关键结果汇总

图 4 进一步汇总了若干关键准确率。可以看到，模型在达到 80% 之后仍有约 5 个百分点的提升空间，但后期提升速度明显变慢。这符合 ViT 在小图像分类任务中的常见训练现象：模型容量足够，但数据规模有限，后期收益主要来自学习率退火、EMA 平滑和正则化带来的泛化改善。若继续提高结果，单纯延长训练未必高效，更值得尝试的是更系统的数据增强组合或调整 patch 大小、embedding 维度和 DropPath 强度。

5 总结

本次实验基于 PyTorch 实现了一个用于 CIFAR10 图像分类的 ViT 模型，完成了数据加载与增强、Patch Embedding、Transformer Encoder、MLP 分类头、训练评估和日志可视化等流程。最终日志中最高测试准确率为 85.79%，验证集最优模型测试准确率为 85.61%，均超过实验要求的 80%。

从实验过程看，ViT 在 CIFAR10 上的关键并不只是堆叠 Transformer 层，而是要用合适的 patch 粒度和正则化策略弥补其缺少卷积归纳偏置的问题。本实验选择 4×4 patch，使每张图像保留 64 个 patch token，既不会像过大 patch 那样丢失过多局部细节，也不会像过小 patch 那样显著增加序列长度和训练开销。训练中 Mixup、Random Erasing、DropPath、EMA 和 warmup-cosine 学习率调度共同保证了泛化表现。

通过本次实验，我进一步熟悉了 ViT 将图像划分为 patch 并转化为 token 序列的建模方式，也实践了多头自注意力、位置编码、残差连接、DropPath、Mixup、EMA 和学习率调度等训练技术。后续如果继续优化，可以尝试 RandAugment 或 CutMix 等更强的数据增强方法，比较不同 patch 大小对局部细节建模的影响，或者在保持参数量可控的前提下调整 embedding 维度和 Transformer 深度，以进一步提升 CIFAR10 分类准确率。